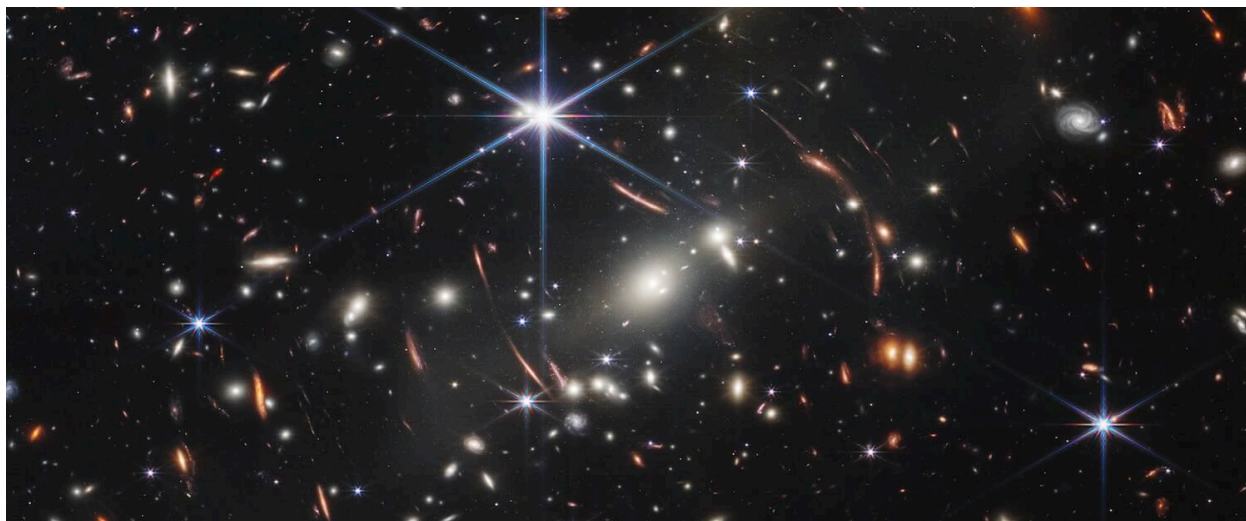




JS. Lesson 2

Ще про hoisting

IIFE

undefined VS undeclared

Області видимості: лексична і динамічна

this

Замикання

Новий синтаксис

1) BigInt

```
var bigNumber1 = new BigInt(329392);  
var bigNumber2 = 323n;  
  
typeof bigNumber1; // 'bigint'
```

Hoisting - var і function

Hoisting застосовується до: **var** і **function declaration**. При цьому повторний var ігнорується, а повторний function declaration перезаписує попередній.

```

var a1 = 2;
var a1 = 3;

function f1() { console.log('f1') }
function f1() { console.log('overriden f1') }

console.log(a1); // 3
f1(); // "overriden f1"

```

let, const, function expression

Hoisting **не** застосовується до: let, const і **function expression**

```

console.log(a2); // error
let a2 = 3;

```

```

f2(); // error
(function f2() {});

```

IIFE - Instantly Invoked Function Expression

Широко вживаний в JS паттерн для виконання коду в ізольованій області, без ризику випадково змінити щось в глобальній області.

```

(function () {
    // isolated namespace
})();

```

undefined VS "undeclared"

В JS існує неоднозначність що саме означає undefined. Задекларована змінна без значення (a) і незадекларована змінна (b) мають один тип - "undefined".

```
var a;  
typeof a; // "undefined"  
typeof b; // "undefined"
```

При цьому звернення до першої повертає undefined, а звернення до другої призводить до помилки.

```
var a;  
a; // undefined  
b; // ReferenceError: b is not defined
```

catch як область видимості

Насправді не лише функції створюють нову область видимості. Змінні, що "кидаються" за допомогою throw ізольовані в блоці catch.

```
var a = 2;  
try {  
  throw 42;  
} catch (a) {  
  var a = 1; // Локальна `a` "затінює" глобальну.  
}  
console.log(a); // 2
```

```
try {  
  throw 42;
```

```
// Зверніть увагу, тут значення перехоплюється з іншим ідентифікатором.  
} catch (err) {  
  var a = 1; // А тут локальної `a` немає, тому зміниться глобальна.  
}  
console.log(a); // 1
```

Лексична область видимості - визначається в момент лексичного розбору коду.

```
function loggerFactory() {  
  var a = {prop: 'inner a'};  
  
  return function logger() {  
    console.log(a.prop);  
  }  
}  
  
var a = {prop: 'outer a'};  
var logger = loggerFactory();  
logger(); // 'inner a'
```

В JS область видимості кожної функції визначається тим, де вона описана, а не тим де викликається.

Динамічна область видимості - визначається в момент виконання коду.

```
function loggerFactory() {  
  return function logger() {  
    console.log(this.prop);  
  }  
}
```

```

}

var a = {
  prop: 'prop of a',
  logger: logger
}

var b = {
  prop: 'prop of b',
  logger: logger
}

window.prop = 'prop of global!';

var logger = loggerFactory();
logger(); // 'prop of global!'
a.logger(); // 'prop of a'
b.logger(); // 'prop of b'

```

В JS значення `this` для кожної функції визначається тим як вона викликається, а не тим де вона описана.

this

Є 4 правила за якими JS визначає поточне значення `this`. Наприклад, маємо функцію:

```

function thisLogger(message, code) {
  console.log(this, message, code);
}

```

При звичайному виклику функції, `this` в ній вказує на глобальний об'єкт - `window`` (в браузері) або `global` (в Node.js). Але при "use strict" буде `undefined`

```
thisLogger('hi!', 0);
```

При виклику функції на об'єкті (як метод), this в ній вказує на цей об'єкт:

```
var o1 = { prop: 'obj1 prop', logger: thisLogger };  
o1.logger('hi', 0);
```

Можна явно "застосувати" функцію до конкретного об'єкта:

```
var o2 = { prop: 'obj2 prop' };  
logger.apply(o2, ['hi', 0]);  
// or  
logger.call(o2, 'hi', 0);
```

При виклику функції в якості конструктора (з приставкою new), this вказує на новостворений об'єкт, який буде повернуто:

```
var o3 = new thisLogger('hi', 0);
```

"Загублений this"

```
var slowMessenger = {  
  msg: 'hi!',  
  sayMsg: function() {  
    setTimeout(function() {  
      // всередині цієї функції `this` більше не вказує  
      на наш об'єкт!    }, 1000);  
  }  
};
```

```
        console.log(this.msg);
    }, 1000);
}
}

slowMessenger.sayMsg(); // 'undefined' (спроба отримати поле
`msg` глобального об'єкта)
```

Часта проблема - втрата this при обгортанні коду в функцію.

Раніше її вирішували за допомогою

JavaScript

Copy

```
var that = this;
```

а зараз в нагоді стануть "стрілкові" функції.

"Lexical this"

Особливістю т. зв. "стрілкових функцій" є те що this в них працює за правилами лексичної області видимості. Тобто, завжди вказує на той самий об'єкт, на який вказує this в поточній лексичній області видимості. Це корисно для уникнення ситуації "загубленого this"

JavaScript

Copy

```
var slowMessenger = {
  msg: 'hi!',
  sayMsg: function() {
    setTimeout(() => {
      console.log(this.msg);
    }, 1000);
  }
}
```

```
slowMessenger.sayMsg(); // 'hi!'
```

Замикання

Замикання - це здатність функції пам'ятати ("замикати" в собі) своє лексичне оточення протягом всього періоду існування.

JavaScript

Copy

```
function outer() {  
    var a = 1;  
    setTimeout(() ⇒ a = 3, 1000);  
    return function inner() { // `inner` завжди матиме доступ до `a`  
        console.log(a);  
    }  
}
```

```
var innerFunc = outer();  
setTimout(innerFunc, 2000);  
innerFunc();
```

Олдскульний JS модуль

JavaScript

Copy

```
var module = function () {  
    var privateVar = 'i am private';  
    return {  
        setPrivateVar: function (val) {  
            privateVar = val.toUpperCase();  
        },  
        getPrivateVar: function () {  
            return privateVar;  
        }  
    }  
}
```



```
    }  
}
```

```
var m = module();
```

```
m.setPrivateVar('private var set with setter');  
console.log(m.getPrivateVar());
```

Замикання в циклі

Поширена проблема - замикання в циклі. Такий код не працює як очікується, тому що на момент виконання функції timer змінна i вже має значення 6.

JavaScript

Copy

```
// так працюватиме, але не вірно  
for (var i=1; i<=5; i++) {  
    setTimeout( function timer(){  
        console.log( i );  
    }, i*1000 );  
}  
// 6 6 6 6 6
```

Рішення - створювати область видимості для кожної ітерації і зберігати поточне значення i в цій області (створюючи нову змінну). Таким чином, кожна з п'яти функцій-timer'ів буде "замикати" в собі коректне значення, замість доступу до глобальної i.

JavaScript

Copy

```
// а так працюватиме вірно  
for (var i=1; i<=5; i++) {  
    (function () {  
        var j = i;  
        setTimeout( function timer(){
```

```

        console.log( j );
    }, i*1000 );
    })()
}
// 1 2 3 4 5

```

let в циклі

Значно спростити код з попереднього слайду можна за допомогою let - так ми створюємо нову область видимості на кожній ітерації циклу, без необхідності обгортання в функцію

JavaScript

Copy

```

// так теж працює вірно
for (var i=1; i<=5; i++) {
    let j = i;
    setTimeout( function timer(){
        console.log( j );
    }, j*1000 );
}

```

Крім того, let має спеціальну поведінку для циклів: якщо декларувати i за допомогою let замість var, то вона буде неявно "переприсвоюватись" на кожній ітерації. Тобто код можна ще більше спростити.

JavaScript

Copy

```

// теж працює вірно (зверніть увагу: `var i=1` замінено на `let i=1`)
for (let i=1; i<=5; i++) {
    // можна уявити що в цьому місці неявно відбувається `let i = i`;
    setTimeout( function timer(){
        console.log( i );
    }, i*1000 );
}

```

Новий синтаксис

Spread, Rest, Destructurization...

В JS з'являється багато нових можливостей і нового синтаксису. Корисний сайт де можна подивитись список нововведень за роками і дізнатись чи підтримуються вони вашою програмою-рантаймом:

<https://compat-table.github.io/compat-table/es6>

Polyfilling VS Transpiling

Коли якісь нововведення ще не підтримуються, в середовищі JS використовують наступні підходи:

Поліфіли - реалізація нових можливостей самостійно (зазвичай достатньо підключити вже готові бібліотеки).

Транспіляція - автоматичне перетворення коду на такий що не покладається на новий синтаксис (використовуються спеціальні програми-транспілятори, наприклад Babel)

Приклад поліфілу:

JavaScript

Copy

```
// допустимо, метод isNaN ще не підтримується JS
Number.isNaN(5); // Uncaught TypeError: Number.isNaN is not a function
```

```
// напишемо його самі!
```

```
if (!Number.isNaN) {
    Number.isNaN = function isNaN(x) {
        return x !== x;
    };
}
```

```
// працює
```

```
Number.isNaN(5); // false
```

Приклад транспіляції:

```
// До транспіляції - код використовує нову можливість JS -  
// значення для аргументів за замовчуванням  
function foo(a = 2) {  
    console.log( a );  
}  
  
foo();      // 2  
foo( 42 );  // 42  
  
// Після транспіляції -  
// код працює так само, але переписаний без використання ново  
// их можливостей JS  
function foo() {  
    var a = arguments[0] !== (void 0) ? arguments[0] : 2;  
    console.log( a );  
}
```

Приклад того як транспілятор міг імітувати блочну область видимості (коли ще не було let):

```
// До транспіляції - `a` доступна лише в межах блоку, але не  
// ззовні  
{  
    let a = 2;  
    console.log(a); // 2  
}  
console.log(a); // ReferenceError!  
  
// Після транспіляції - поведінка та сама, але без використан  
// ня `let` -  
// використовується той факт що змінні "кинуті" і перехоплені  
// за допомогою catch
```

```
// ізолювані в межах блоку catch
try {
    throw 2;
} catch (a) {
    console.log(a); // 2
}
console.log(a); // ReferenceError!
```